

UNIVERSIDAD DEL VALLE DE GUATEMALA
CC3067 - Redes
Sección 20
Ing. Jorge Yass



Laboratorio 3 - Definición de protocolo

Osman de León - 23428
Oliver Viau - 23544

Guatemala, 13 de agosto del 2026

Descripción de los Algoritmos Utilizados y su Implementación

Protocolo Link-State

Cada router solo conoce a sus vecinos directos, que están definidos en su archivo de configuración JSON. Para comprobar que un enlace sigue activo, cada router envía un mensaje HELLO a cada vecino cada `HELLO_INTERVAL = 4` segundos. El vecino que recibe el HELLO responde con un HELLO_ACK. Si un router no recibe un HELLO_ACK de un vecino en `HELLO_TIMEOUT = 12` segundos, el enlace se marca como caído. Así funciona el mecanismo de detección de fallas del protocolo.

Cuando cambia el conjunto de enlaces activos de un router (ya sea por una caída o una recuperación), el router construye un nuevo Link State Advertisement (LSA). Este LSA contiene su identificador de origen, un número de secuencia incremental y la lista de vecinos activos con su costo asociado. El LSA se propaga mediante flooding: cada router que recibe un LSA lo reenvía a todos sus vecinos, excepto a quien se lo envió, y solo si el número de secuencia recibido es mayor al último que conoce para ese origen. Esta regla evita que los LSAs se reenvíen en ciclos infinitos y también evita que se procese información que ya está obsoleta. La colección de LSAs que recibe cada router forma su base de datos de estado de enlace (LSDB), que representa su vista del grafo completo de la red.

Algoritmo de Dijkstra

A partir de la LSDB, cada router construye un grafo dirigido con pesos ($\text{origen} \rightarrow \{\text{vecino: costo}\}$) y ejecuta el algoritmo de Dijkstra tomándose a sí mismo como nodo fuente. La implementación utiliza una cola de prioridad (heapq) para seleccionar en cada iteración el nodo no visitado con la distancia acumulada mínima, y relaja las distancias de sus vecinos si se encuentra un camino más corto. Una vez calculadas las distancias mínimas a todos los routers alcanzables, se reconstruye, para cada destino, el primer salto (`next_hop`) retrocediendo por el arreglo de predecesores (`prev`) hasta encontrar el vecino directo del router local. El resultado se materializa como una tabla de enrutamiento en memoria y se exporta a un archivo `<nodo>_tabla_enrutamiento.csv` con las columnas `destino`, `siguiente_salto`, `ip`, `puerto` y `costo`. Es importante notar que las tablas de enrutamiento sólo contienen rutas hacia otros routers: los clientes y servidores no participan del cálculo de Dijkstra ni se anuncian en los LSA.

Enrutamiento hacia clientes y servidores

Dado que los LSA sólo contienen información entre routers, la relación entre un endpoint final (por ejemplo, `servidor1`) y su router gateway se resuelve mediante configuración local (`endpoint_routes`), la cual nunca se transmite por la red ni forma parte del protocolo compartido con otras implementaciones. Cuando un router recibe un MESSAGE cuyo campo `to` no corresponde a un endpoint conectado localmente (`local_endpoints`), consulta `endpoint_routes` para determinar cuál router funciona como gateway del destino, y luego consulta su tabla de enrutamiento (previamente calculada por Dijkstra) para obtener el siguiente salto hacia ese router gateway. El campo `to` del mensaje nunca se modifica durante el recorrido; solamente cambia el receptor inmediato al que se reenvía el paquete.

Codificación Hamming(7,4)

Para simular un canal con ruido entre routers, cada mensaje JSON que viaja en un salto router-router se codifica con Hamming(7,4). El código se aplica en bloques de 4 bits de datos (d1, d2, d3, d4), a los que se agregan 3 bits de paridad (p1, p2, p3) calculados mediante XOR sobre subconjuntos definidos de los bits de datos, siguiendo el orden acordado con las demás parejas del curso: posiciones 1, 2, 4 para los bits de paridad y 3, 5, 6, 7 para los bits de datos. En la decodificación, se recalculan tres bits de chequeo (c1, c2, c3) cuya combinación forma un síndrome de 3 bits: un síndrome distinto de cero indica la posición exacta (1-indexada) del bit alterado, permitiendo invertirlo y así recuperar la palabra código original. El texto (JSON en UTF-8) se convierte primero a una secuencia de bits múltiplo de 8, por lo que siempre es también múltiplo de 4 y no requiere relleno (padding) adicional antes de fragmentarse en nibbles de 4 bits.

Cabe destacar que Hamming(7,4) es un código SEC (Single Error Correction): garantiza la corrección de, como máximo, un bit erróneo por cada bloque de 7 bits. La sección de Resultados presenta evidencia empírica de este límite: con un bit de error por bloque el mensaje se recupera siempre correctamente, mientras que con dos bits de error dentro del mismo bloque el algoritmo corrige hacia una palabra código incorrecta, degradando la información en lugar de repararla.

Arquitectura de concurrencia y comunicación

Cada router (node.py) ejecuta cuatro tareas concurrentes mediante threading.Thread: (1) un servidor de sockets que acepta conexiones entrantes y despacha cada línea recibida a un manejador, (2) un ciclo que envía HELLO periódicamente a los vecinos, (3) un monitor de liveness que revisa cada `MONITOR_INTERVAL = 3` segundos si algún vecino superó el `HELLO_TIMEOUT`, y (4) el hilo principal, que permanece a la espera de la señal de interrupción (Ctrl+C). El acceso a estructuras compartidas (LSDB, tabla de enrutamiento, estado de vecinos) se protege con un `threading.Lock`. La comunicación utiliza sockets TCP con mensajes delimitados por un carácter de salto de línea (`\n`) y codificados en UTF-8; el receptor distingue si el contenido es JSON válido (HELLO, HELLO_ACK, LSA, MESSAGE) o una cadena de bits Hamming proveniente de otro router. El cliente (client.py) y el servidor (server.py) no participan del Link State: el cliente sólo conoce a su router gateway y envía JSON plano; el servidor únicamente escucha en su puerto a la espera de que su router gateway le entregue el mensaje final, también en JSON plano.

Resultados

Los resultados presentados a continuación se obtuvieron ejecutando directamente el código fuente entregado, sobre la topología de prueba local definida en la documentación del proyecto:

```
cliente1 — A —(2)— B —(3)— C — servidor1
```

con servidor1 configurado como endpoint local de C, y cliente1 como endpoint local de A. Todos los procesos se ejecutaron en 127.0.0.1 con los puertos 6001 (A), 6002 (B), 6003 (C), 9001 (cliente1) y 9002 (servidor1).

Convergencia del protocolo y tablas de enrutamiento

Tras iniciar los tres routers y esperar el intercambio de HELLO/HELLO_ACK y la propagación de LSA por flooding, cada router calculó mediante Dijkstra su tabla de enrutamiento y la exportó a un archivo CSV. Las tablas obtenidas fueron las siguientes:

Router	Destino	Siguiente salto	IP	Puerto	Costo
A	B	B	127.0.0.1	6002	2
A	C	B	127.0.0.1	6002	5
B	A	A	127.0.0.1	6001	2
B	C	C	127.0.0.1	6003	3
C	A	B	127.0.0.1	6002	5
C	B	B	127.0.0.1	6002	3

Estos resultados coinciden exactamente con el cálculo teórico esperado: para la topología A–(2)–B–(3)–C, la ruta más corta de A hacia C tiene costo $2 + 3 = 5$ y pasa obligatoriamente por B, lo cual confirma que la implementación de Dijkstra y la reconstrucción del siguiente salto son correctas.

Envío de un mensaje cliente → servidor

Con las tres tablas de enrutamiento ya calculadas, se envió desde cliente1 el mensaje MESSAGE con payload "Hola Servidor" dirigido a servidor1. El recorrido observado en los logs de cada nodo fue el siguiente:

```
[A] MESSAGE from=cliente1 to=servidor1 hops=1 payload='Hola Servidor'
[B] MESSAGE from=cliente1 to=servidor1 hops=2 payload='Hola Servidor'
[C] MESSAGE from=cliente1 to=servidor1 hops=3 payload='Hola Servidor'
[servidor1] Mensaje de cliente1 (hops=3): Hola Servidor
```

El contador hops se incrementó exactamente una vez por cada router que procesó el mensaje (A, B y C), y el servidor recibió el payload sin alteraciones, confirmando que: (a) el tramo cliente → gateway viaja en JSON plano sin Hamming, (b) los tramos router → router se codifican y decodifican correctamente con Hamming(7,4) sin introducir errores en ausencia de ruido simulado, y (c) el último tramo gateway → servidor también viaja en JSON plano, tal como especifica el protocolo.

Detección de caída y recuperación de enlaces

Con la red en funcionamiento, se detuvo el proceso del router B para simular una falla. Aproximadamente 12 segundos después (HELLO_TIMEOUT), tanto A como C —vecinos directos de B— marcaron el enlace como caído y regeneraron su LSA sin ese enlace:

```
[A] Enlace con B marcado como CAIDO (sin HELLO_ACK en 12s).
[C] Enlace con B marcado como CAIDO (sin HELLO_ACK en 12s).
```

Al reiniciar el proceso de B, este comenzó a responder nuevamente a los HELLO periódicos de A y C, quienes detectaron la recuperación del enlace y regeneraron sus LSA, disparando un nuevo cálculo de Dijkstra:

```
[A] Enlace con B recuperado.
[C] Enlace con B recuperado.
```

Este comportamiento confirma que el mecanismo de liveness (HELLO/HELLO_ACK + monitor cada 3 segundos) detecta correctamente tanto la caída como la recuperación de un enlace, y que ambos eventos disparan la regeneración y re-flooding del LSA, seguido de un nuevo cálculo de rutas.

Comportamiento de Hamming(7,4) ante ruido simulado

Se codificó un mensaje MESSAGE representativo (97 caracteres, 776 bits de datos) y se evaluó la decodificación bajo tres escenarios: sin ruido, con exactamente un bit de error introducido aleatoriamente en cada uno de los 194 bloques de 7 bits, y con dos bits de error dentro de un mismo bloque.

Escenario	Bloques (7 bits)	Errores corregidos	Mensaje recuperado correctamente
Sin ruido	194	0	Sí
1 bit de error por bloque	194	194	Sí
2 bits de error en un mismo bloque	1 de 194	1 (incorrecto)	No — UnicodeDecodeError

En el escenario con un bit de error por bloque, la longitud codificada fue de 1358 bits (overhead de 1.75x sobre los 776 bits originales) y el mensaje se reconstruyó de forma idéntica al original en el 100% de los 194 bloques. En el escenario con dos bits de error dentro de un mismo bloque, el síndrome calculado apuntó a una posición de bit incorrecta: el algoritmo "corrigió" un bit que no estaba dañado, produciendo una palabra código diferente a la original y, en este caso particular, bytes que no forman una secuencia UTF-8 válida (UnicodeDecodeError: invalid start byte). Esto confirma empíricamente el límite teórico de Hamming(7,4) como código SEC de un solo bit.

Discusión

Los resultados obtenidos validan que la implementación cumple con los objetivos funcionales del laboratorio: descubrimiento de vecinos, propagación de estado de enlace, cálculo de rutas óptimas, entrega extremo a extremo de mensajes de aplicación, tolerancia a fallas de enlace y corrección de errores de un bit en el canal simulado.

Un aspecto de diseño que resultó particularmente relevante es la separación estricta entre la información que forma parte del protocolo compartido (HELLO, HELLO_ACK, LSA y MESSAGE, cuyo formato debe ser interoperable con las implementaciones de otras parejas del curso) y la información puramente local de esta implementación (endpoint_routes y local_endpoints). Esta separación permite que cada pareja resuelva el mapeo endpoint → gateway con su propia lógica interna sin comprometer la compatibilidad de los mensajes que efectivamente viajan por la red. Esto es consistente con el principio de diseño en capas de los protocolos de red, donde el plano de control (Link State) y las decisiones locales de forwarding se mantienen desacoplados.

El uso de un número de secuencia por origen en los LSA, junto con la regla de "descartar si $seq \leq$ conocido", resuelve de forma simple el problema clásico de los bucles de flooding en protocolos Link-State, evitando que la red se sature con mensajes redundantes o desactualizados. Sin embargo, esta implementación no incorpora un tiempo de expiración (aging) para los LSA ni retransmisión ante pérdida de un LSA durante el flooding. En una red con pérdida de paquetes más agresiva que la simulada aquí (sockets TCP locales, que son confiables), routers aislados temporalmente podrían quedar con una vista desactualizada del grafo hasta recibir el siguiente LSA del origen correspondiente.

Respecto a Hamming(7,4), los resultados muestran su idoneidad para canales con baja probabilidad de error de bit, pero también su fragilidad ante ráfagas de error (burst errors) o ante una tasa de error por bit (BER) suficientemente alta como para que ocurran dos o más errores dentro de una misma palabra código de 7 bits. En ese caso, el mecanismo no solo falla en corregir, sino que puede introducir corrupción silenciosa: el síndrome no distingue entre "cero errores" y "un patrón particular de dos errores", como se observó en la prueba con dos bits de error. Esto es consistente con la teoría de códigos de bloque lineales, donde la distancia mínima de Hamming(7,4) es 3, lo que garantiza detección de hasta 2 errores pero corrección confiable de solo 1 error.

Conclusiones

Se implementó exitosamente un protocolo de enrutamiento Link-State completo —incluyendo descubrimiento de vecinos, flooding de LSA, cálculo de rutas mediante Dijkstra y entrega de mensajes extremo a extremo— sobre una arquitectura concurrente basada en sockets TCP y threading en Python, sin dependencias externas.

Las pruebas realizadas confirman que el sistema calcula correctamente las rutas de menor costo, detecta y se recupera de caídas de enlace en los tiempos configurados, y entrega mensajes de aplicación de cliente a servidor a través de múltiples saltos de router, incrementando correctamente el contador de saltos en cada tramo.

La codificación Hamming(7,4) implementada corrige de forma confiable un bit de error por cada bloque de 7 bits, tal como establece la teoría, pero no ofrece garantías ante dos o más errores en un mismo bloque, lo cual quedó demostrado empíricamente. Esto resalta la importancia de dimensionar el mecanismo de corrección de errores de acuerdo con las condiciones reales del canal de comunicación que se desea modelar o proteger.

Comentarios

Durante el desarrollo, uno de los mayores retos fue acordar y respetar estrictamente el formato de los mensajes del protocolo (HELLO, HELLO_ACK, LSA, MESSAGE) con las demás parejas del curso, dado que pequeñas diferencias de nomenclatura o de orden de los bits de paridad en Hamming(7,4) habrían roto la interoperabilidad entre implementaciones distintas. Mantener la información específica de esta implementación (endpoint_routes, local_endpoints) completamente fuera del protocolo compartido resultó clave para lograr dicha compatibilidad sin sacrificar flexibilidad interna.

Referencias

- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271. <https://doi.org/10.1007/BF01386390>
- Hamming, R. W. (1950). Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2), 147–160. <https://doi.org/10.1002/j.1538-7305.1950.tb00463.x>
- Moy, J. (1998). OSPF version 2 (RFC 2328). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc2328>
- Python Software Foundation. (2024). Threading — Thread-based parallelism. Python 3 documentation. <https://docs.python.org/3/library/threading.html>
- ChatGPT (2026). (Corrección de errores y formato para el reporte)